Why Code?

1 An unfinished job

Bad tape, a reading error, or a changed stored value can make a computation fail. For example, a missing hole changes an intended 1 into a 0.

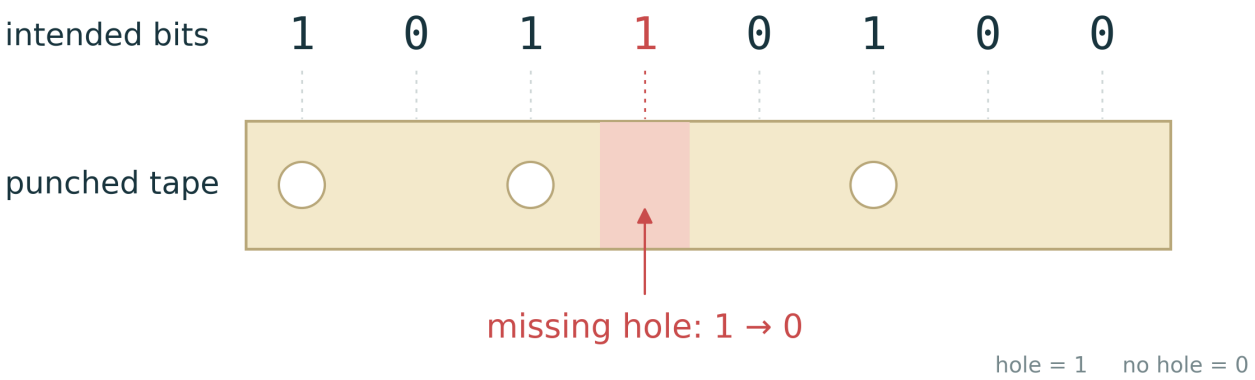


Figure 1: The intended bits are shown for comparison. The machine does not know them; it must check the recording using an agreed rule.

When a check detects an error, the machine can try again. A temporary reading or computation fault may disappear on a retry. Re-reading alone cannot restore a missing hole. One remedy is to repair or replace the tape.

If repeated attempts fail, skip the current job and continue with the next one. Hamming encountered this over two weekends: after three failed attempts, his machine abandoned the job and moved on.[1]

$$\text{check fails} \longrightarrow \text{retry} \longrightarrow \text{still fails: skip this job.}$$

The machine can continue, but this job is still unfinished. Hamming wanted the machine to locate and correct the error so the job could finish.

First, how can the machine recognize an error?

2 Agree on which recordings are allowed

The machine does not know which word was sent. It knows which words the sender was allowed to send.

For example, agree on four allowed recordings:

$$000, \quad 011, \quad 101, \quad 110.$$

The sender encodes each message as one of these words. A received word outside this list signals an error.

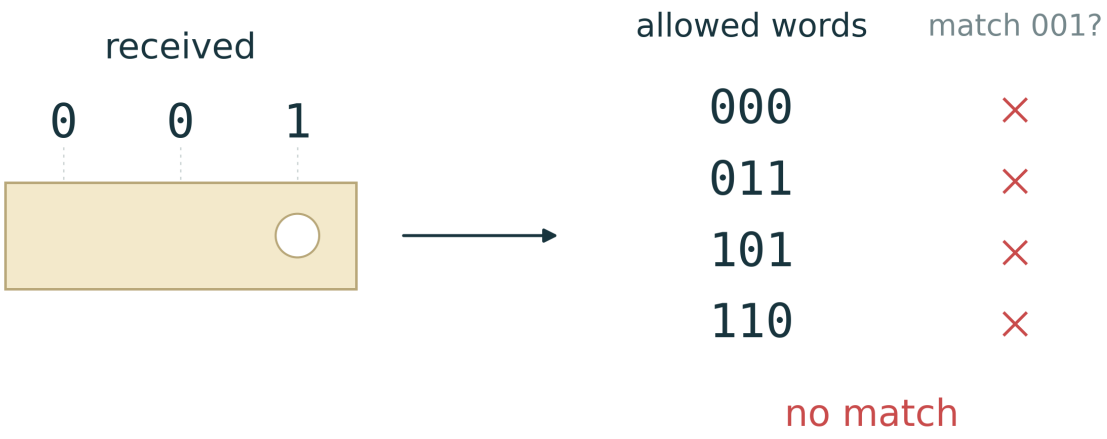


Figure 2: The received 001 matches none of the allowed words. The crosses mark failed comparisons.

Store the four words in a list. Each word has three bits:

$$\underbrace{4}_{\text{words}} \times \underbrace{3}_{\text{bits per word}} = 12 \text{ stored bits.}$$

To reject 001, a sequential search must try all four entries.

In general, storing M words of n bits requires nM bits. A sequential search compares up to M words, checking up to n positions in each. These are costs of this list method; indexing can speed up the search.

A large rulebook for a short input. A 20-bit block has 2^{20} possible patterns. Suppose we allow half:

$$M = \frac{2^{20}}{2} = 524{,}288.$$

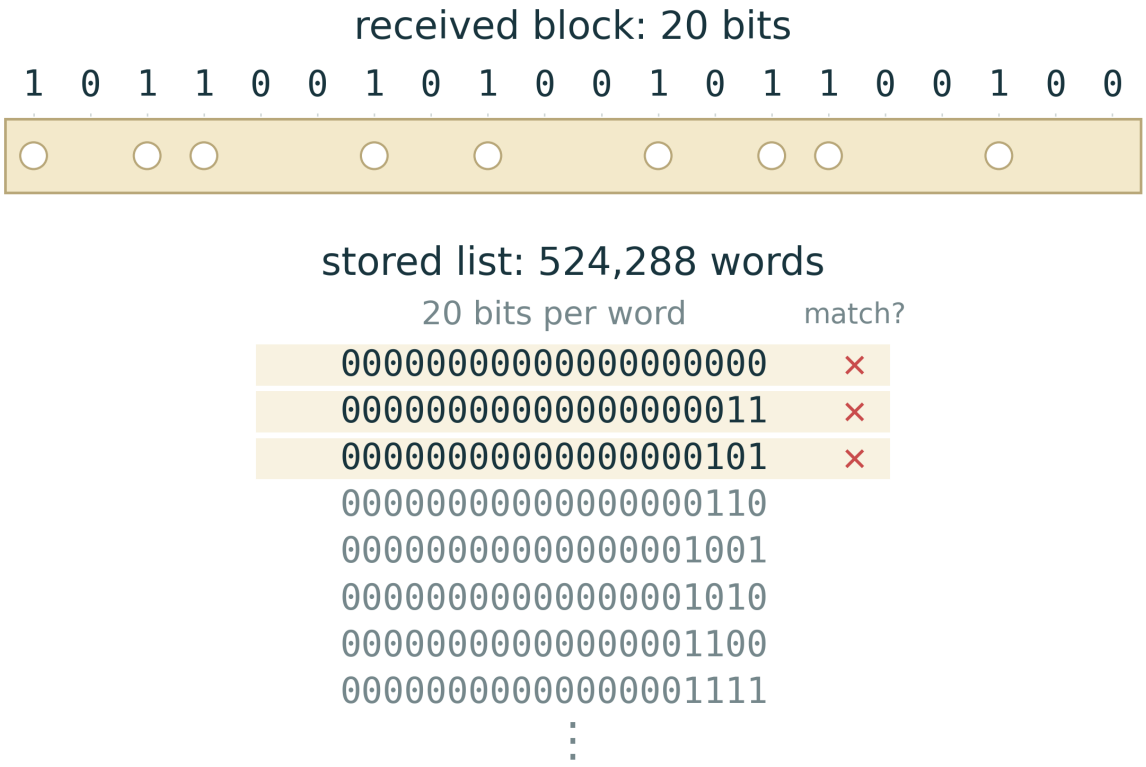


Figure 3: One received block and a large stored list. Only the first few entries are shown.

Storing these words explicitly requires

$$524{,}288 \times 20 = 10{,}485{,}760 \text{ bits.}$$

Suppose we have one million bits for this list. We can store only

$$\frac{1{,}000{,}000}{20} = 50{,}000 \text{ words} < 524{,}288.$$

The list does not fit. Also, for every received word is absent, a sequential search must be operated, which takes a lot of time to do so.

Can we check the received word with a short computation, without storing this large list?

3 Start with the input, then add

Instead of searching the list, compute directly from the received bits. Start with their sum:

$$S = x_1 + \cdots + x_n, \qquad x_i \in \{0, 1\}.$$

Each hole contributes 1. Look at our four allowed recordings:

Recording	S
000	0
011	2
101	2
110	2

Their sums are all even. For three bits, this rule describes exactly the same allowed set:

even S : pass	odd S : fail.

Make the sender follow the rule. Append one check bit. If the data has an odd number of ones, append 1. If it already has an even number, append 0.

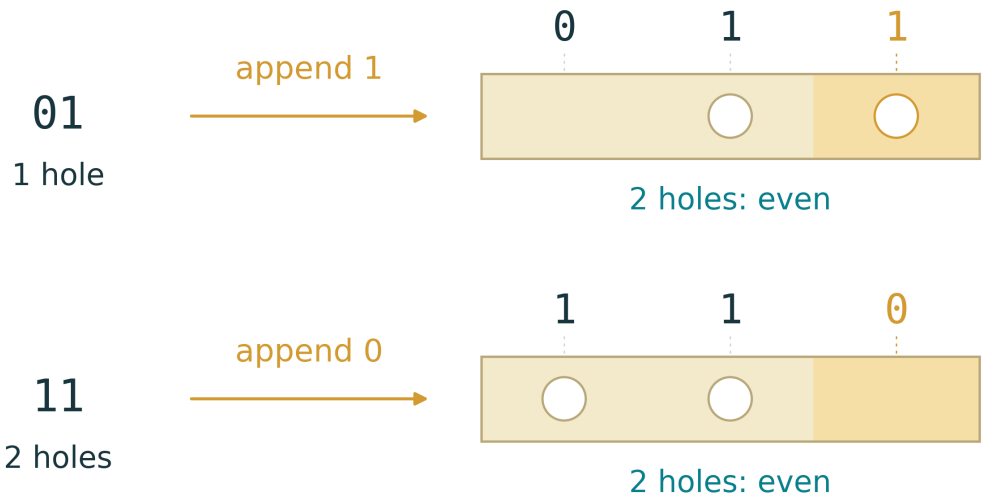


Figure 4: Gold marks the added position. Both choices make the total number of holes even.

$$\underbrace{k}_{\text{data bits}} + \underbrace{1}_{\text{check bit}} = \underbrace{n}_{\text{recorded bits}} .$$

Keep only the parity. We only need to remember even or odd. Reading 0 leaves this state unchanged. Reading 1 switches it.

Use 0 for even and 1 for odd. For the received word `011`:

$$\underbrace{0}_{\text{start}} \xrightarrow{\text{read } 0} 0 \xrightarrow{\text{read } 1} 1 \xrightarrow{\text{read } 1} 0.$$

The ordinary sum is 2; its parity is 0. This two-state addition is arithmetic in $\mathbb{F}_2 = \{0, 1\}$:

$$1 + 1 = 0 \quad \text{in } \mathbb{F}_2.$$

Read all n positions, including the check bit. Keep one bit of parity state, instead of storing the list of allowed words.

One changed bit is detected. A single bit flip changes the ordinary sum by $+1$ or -1. An even sum becomes odd, so the check fails.

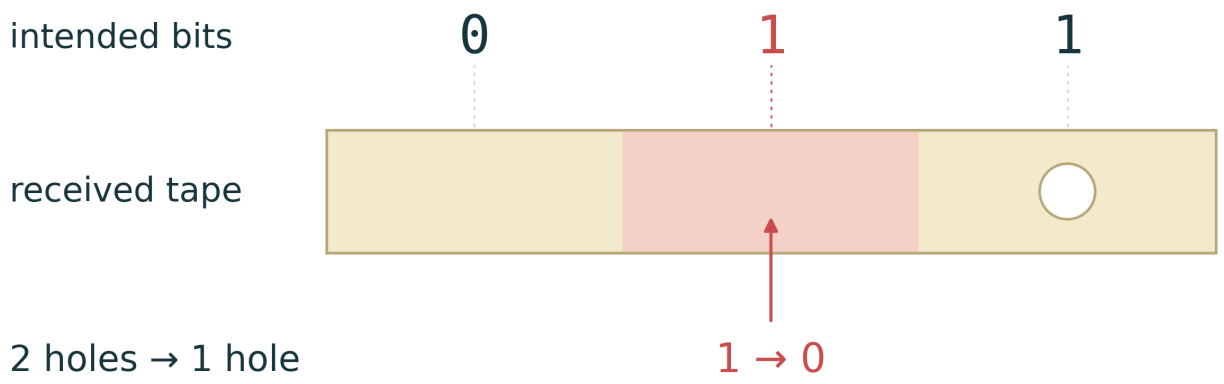


Figure 5: One missing hole changes 011 to 001, and the hole count from two to one.

The machine need not know the original `011`. It sums the received `001` and finds an odd total.

Detection does not guarantee correction. Both `000` and `011` can become `001` after one flip.

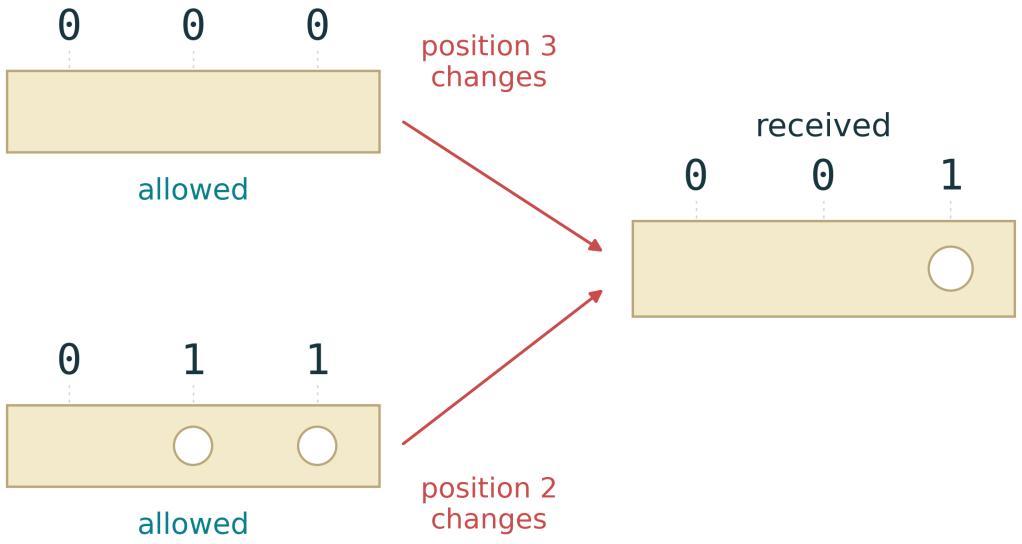


Figure 6: Two allowed originals give the same received word.

The check detects an error, but cannot tell which bit to change. The error process can merge different originals; this does not mean the encoding itself is many-to-one.

Two errors can go undetected. Two added holes change `000` into `011`.

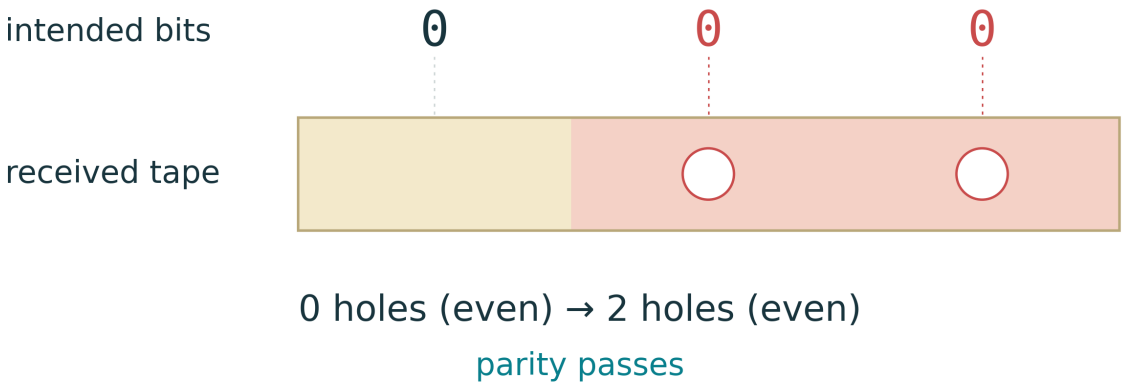


Figure 7: Both sums are even. The recording changed, but the check passes.

Can we use extra positions to separate the allowed recordings further, so that two flips are still detectable?

4 Spend more positions

Choose a new code for one data bit. Record it three times:

$$0 \longmapsto 000, \qquad 1 \longmapsto 111.$$

The two allowed recordings differ in all three positions. Changing only one or two positions cannot turn one into the other.

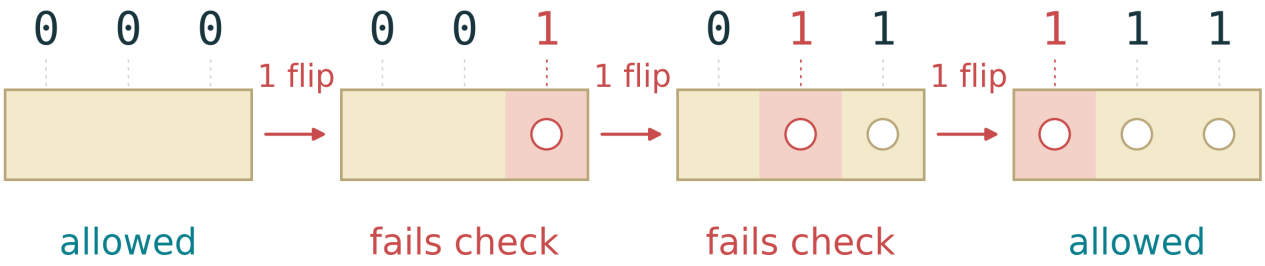


Figure 8: One or two changed positions fail the check. Three changes can reach the other allowed recording.

Correction needs an error bound. The check detects one or two changed bits. To correct an error, assume that at most one bit changed:

$$000 \xrightarrow{\text{1 change}} 001 \xleftarrow{\text{2 changes}} 111.$$

Under this assumption, the received `001` must come from `000`. The majority recovers 0. If two changes are possible, `111` could also produce `001`, and the majority can be wrong.

Compare the same message. For the data `01`, first reuse the preceding section's rule: append one check bit at the end so the total number of ones is even.

$$01 \longmapsto 01\boxed{1} \qquad \text{appended even-parity bit: 3 positions,}$$

$$01 \longmapsto 000\,111 \qquad \text{repeat each data bit: 6 positions.}$$

In general,

$$k \text{ data bits} \longmapsto 3k \text{ recorded bits}, \qquad 2k \text{ extra bits.}$$

For each triple, count the ones: 0 or 3 passes the check. The count $0, 1, 2, 3$ fits in two bits of checking state.

Use a similar idea for decimal digits. Parity uses one check bit for a whole binary block. Try one check digit for a block of decimal digits $0, \ldots, 9$.

The digits $2, 4, 1$ sum to 7. Append 3 to reach 10. If the sum is already 10, append 0.

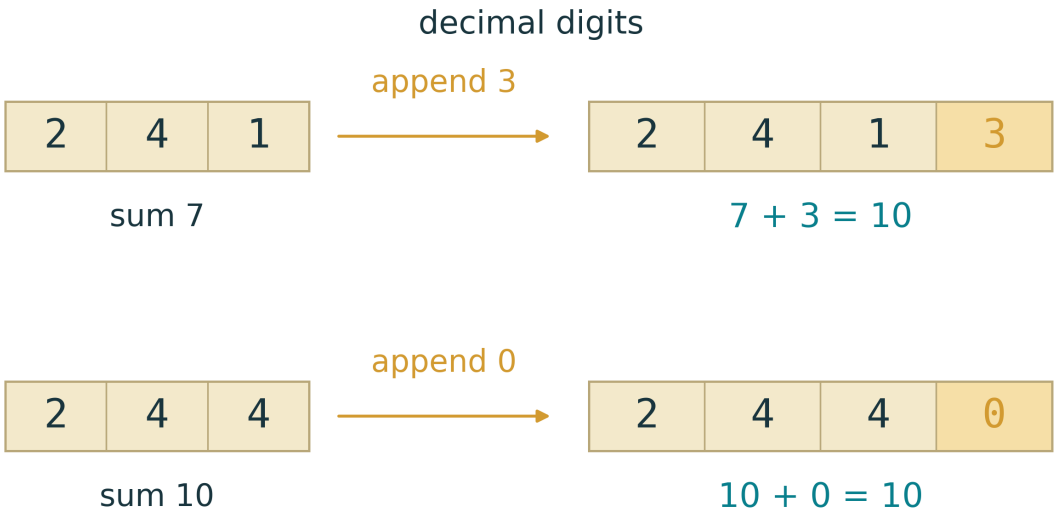


Figure 9: Each cell is a decimal digit, not a bit. Gold marks the appended check digit.

The target is not only 10: for example, $17 + 3 = 20$. The rule is

the total sum is a multiple of 10.

The choices $0, \ldots, 9$ can complete any remainder after division by 10.

Cost and limits. We add one decimal digit per block. Changing one digit changes the sum by a nonzero amount between -9 and 9, so the check fails. Two changes can cancel:

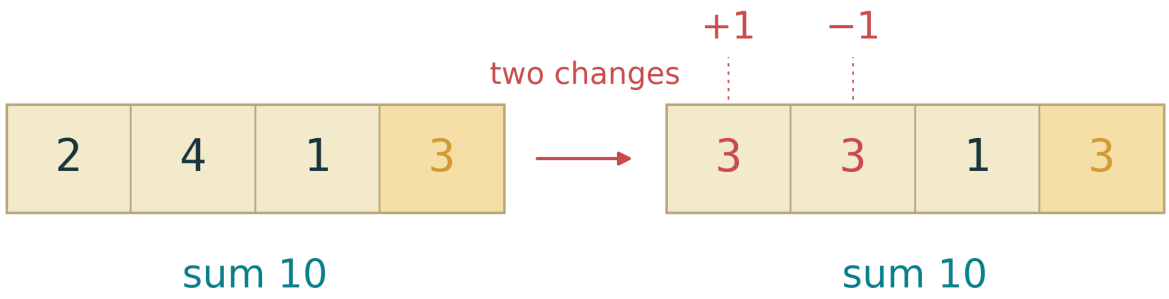


Figure 10: The changes $+1$ and -1 cancel. Both totals are 10, so the changed recording still passes.

Swapping two digits also leaves the sum unchanged. This check does not generally tell us how to correct an error.

One decimal digit needs several tape positions. Suppose we give every decimal digit its own binary pattern of the same fixed length. Three bits offer only eight patterns. We need at least four:

$$2^3 = 8 < 10 \leq 16 = 2^4.$$

Choose ten distinct four-bit patterns, one for each digit. The six unassigned patterns are invalid. Here is one possible assignment for the digits in our example:

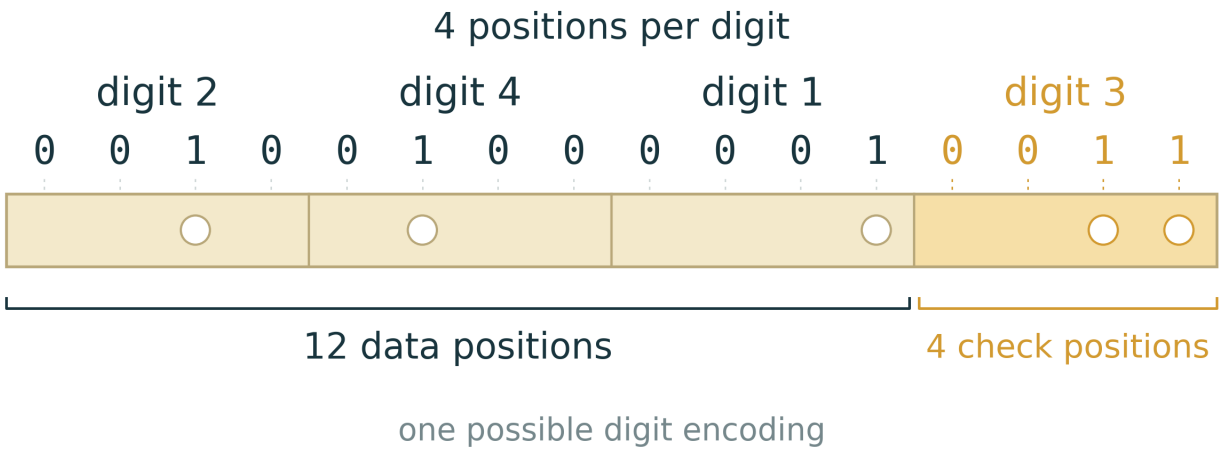


Figure 11: Four positions per decimal digit. The last four positions store the check digit 3. The group boundaries are fixed.

With this encoding, three data digits occupy twelve tape positions. One check digit adds four more:

$$\underbrace{3 \cdot 4}_{\text{data positions}} + \underbrace{1 \cdot 4}_{\text{check positions}} = 16 \text{ positions.}$$

Thus one decimal check digit costs four extra binary positions on this tape. During checking, the running remainder $0, \ldots, 9$ also fits in four bits of state; that is a separate memory cost.

Barcodes use a related decimal check. Distinguishing ten digits is only part of their design.

How should we print these patterns so that a scanner can find and read the digits?

5 Give each digit a printed pattern

Four bits can distinguish ten decimal digits. A printed label must also let a scanner find and measure the pattern.

Woodland and Silver explored automatic product identification. Laurer's team at IBM developed UPC, adopted in 1973 and first used at a retail checkout in 1974.[2]

UPC is the general name. Here we study its regular twelve-digit form, *UPC-A* (UPC Version A). Below, "UPC" refers to this form.

Which side of the middle? A middle marker separates two blocks of six digits:

$$\underbrace{\text{six digits}}_{\text{left half}} \quad \boxed{\text{middle}} \quad \underbrace{\text{six digits}}_{\text{right half}}.$$

Left and right refer to positions on the printed label. Read the label from left to right. Each digit has four nonempty runs, with these rules:

digit's position	run colors in order	black-module count
left of middle	white–black–white–black	odd
right of middle	black–white–black–white	even

A *module* is one equal-width position: 0 for white, 1 for black. A run can occupy several modules. The odd/even rule counts black modules, not black runs; every digit has two black runs. This is *character parity*.

Agree on a table. UPC uses seven modules per digit. Its left codebook assigns `0001101` to digit 0:

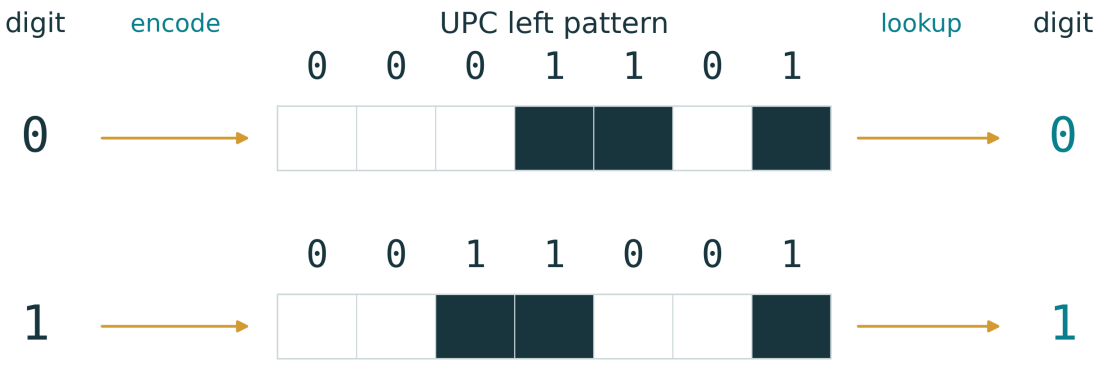


Figure 12: Encoding chooses the assigned pattern. Decoding looks up that same pattern and returns its digit.

The arrow returns 0 because of this agreement. As an ordinary binary numeral, $(0001101)_2$ would instead have value 13.

Each side must represent all ten digits, $0, 1, \ldots, 9$. Each therefore needs *at least ten different patterns*. With fewer, two digits would share a pattern. More than ten is allowed; some patterns could be left unused.

Why is seven the smallest width under these rules? Count the left-side patterns first. Their color order is fixed; we vary the four positive integer widths a, b, c, d:

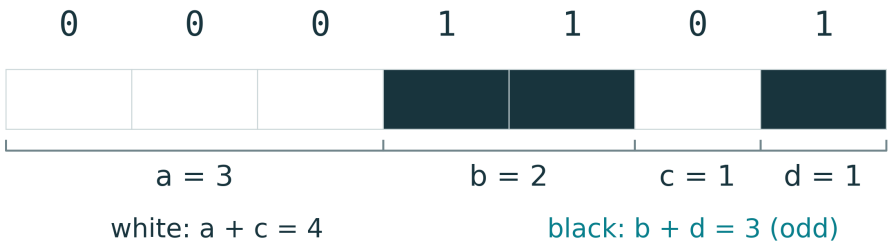


Figure 13: One allowed pattern: widths $(a, b, c, d) = (3, 2, 1, 1)$ give the left-side digit 0.

The total width is $n = a + b + c + d$. Black occupies $B = b + d$ modules and must be odd. White occupies $n - B$. Each color needs at least two modules, one for each of its runs.

Try six modules. Then $B = 3$ and white also totals 3. Black has two splits, $(b, d) = (1, 2)$ or $(2, 1)$. White has two splits, $(a, c) = (1, 2)$ or $(2, 1)$. Every black split can pair with either white split:

$$\underbrace{2}_{\text{black splits}} \cdot \underbrace{2}_{\text{white splits}} = 4 \text{ patterns} < 10.$$

These combinations change the widths. The colors stay in the fixed order white–black–white–black.

Try seven modules. Now the odd black total can be 3 or 5:

black total	white total	number of patterns
3	4	$2 \cdot 3 = 6$
5	2	$4 \cdot 1 = 4$

$6 + 4 = 10.$

Together, these combinations give ten distinct left-side patterns, enough to represent all ten digits.

Could anything shorter work? Four nonempty runs need at least four modules. At width 4, the only left pattern is `0101`, which fails odd parity. At width 5, black must total 3 and white 2, giving $2 \cdot 1 = 2$ patterns:

width n	$n < 4$	4	5	6	7
allowed left patterns	0	0	2	4	10

At width 7, a left pattern has odd B black modules and even $7 - B$ white modules. For pattern on right side, the ten resulting right patterns have order black–white–black–white and an even black count. It has smae one-to-one pattern to digit logic as left pattern but color reverse. (you may think that it will make the bar code looking symmetric, but not! It is a bit flip instead of a flip by middle :-O)

Thus seven is the *minimum width for both sides under these run and parity rules.* Longer patterns are possible. Left and right counts cannot be pooled: each side needs its own ten digit meanings.

Could a barcode be all white or all black? Yes, under a different rule. The repetition code allows $0 \mapsto 000$ and $1 \mapsto 111$. We can print those bits as white or black regions if the block boundaries are supplied:

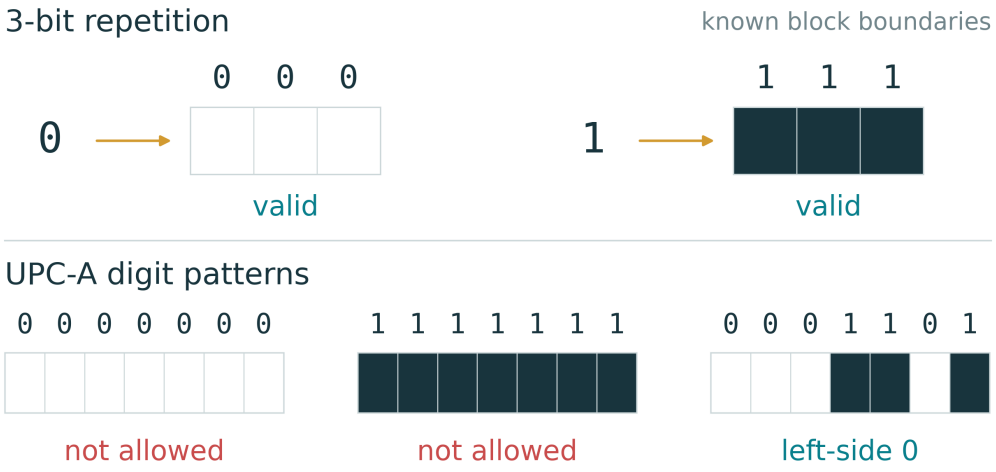


Figure 14: Uniform patterns are valid for repetition. UPC chooses a different set of patterns. The outlines are teaching guides.

UPC requires two white runs and two black runs in every digit. A uniform patch fails that rule. A full UPC label also contains mixed guard patterns, so it cannot be entirely white or black.

Give the scanner more paths. A thin row is easy to miss. Extend the black regions vertically:

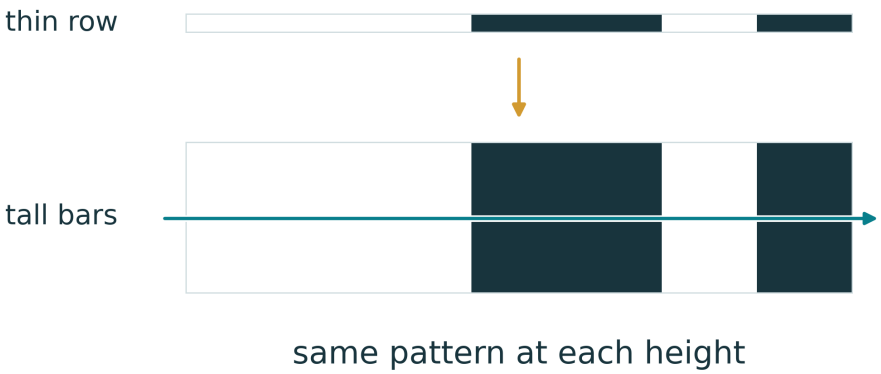


Figure 15: Extra height gives more paths through the same pattern. It costs label area, but adds no digits.

A scan may pass above or below a local scratch.

Read one digit. For now, suppose the boundaries of one seven-module digit are already marked. Move a light spot across it at a locally steady speed. White reflects more light than black.

The sensor measures how long each white or black run lasts:

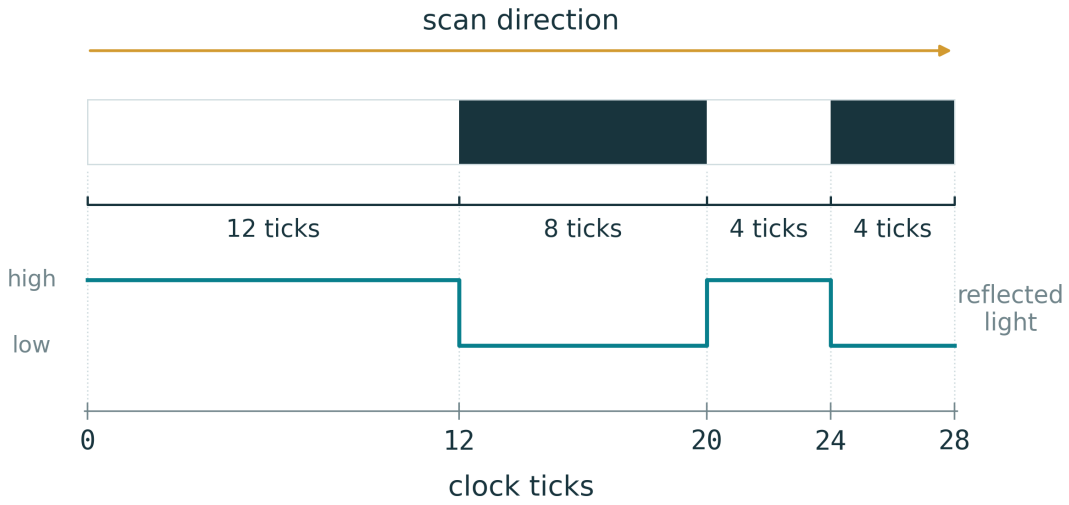


Figure 16: The boundaries $0, 12, 20, 24, 28$ give durations $12, 8, 4, 4$ clock ticks. These are measured times, not bit values.

The whole digit takes 28 ticks and spans seven modules. Its local width unit is

$$u = \frac{12 + 8 + 4 + 4}{7} = 4 \text{ ticks per module.}$$

Convert each measured duration to a width, then expand the runs in their observed color order:

$$(12, 8, 4, 4) \div 4 = (3, 2, 1, 1) \quad \longrightarrow \quad \underbrace{000}_{\text{3 white}}\ \underbrace{11}_{\text{2 black}}\ \underbrace{0}_{\text{1 white}}\ \underbrace{1}_{\text{1 black}}\ .$$

The encoder's table assigned this pattern to digit 0. The reader uses the same table in reverse:

$$\text{digit } 0 \xrightarrow{\text{encode}} \texttt{0001101}, \qquad \texttt{0001101} \xrightarrow{\text{look up}} \text{digit } 0.$$

We supplied the digit boundaries. How can a scanner find them on a real label?

6 Find the boundaries and direction

A scanner may cross nearby text before reaching the barcode. Leave a blank margin on each side: a *quiet zone*.

Then add three known patterns:

$$\underbrace{101}_{\text{start}}, \quad \underbrace{01010}_{\text{middle}}, \quad \underbrace{101}_{\text{end}}.$$



Figure 17: The outer guards and quiet zones mark candidate endpoints. The middle guard supplies an internal boundary.

The short alternating runs provide a width reference. From a candidate start, the reader expects six digits, the middle guard, six more digits, and the end guard:

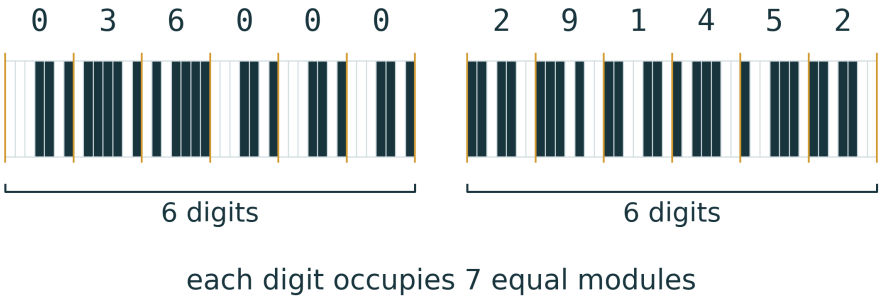


Figure 18: Each digit has four runs spanning seven modules. The reader groups the measured runs and tests their widths.

The structure uses

$$\underbrace{3}_{\text{start}} + \underbrace{6 \cdot 7}_{\text{left digits}} + \underbrace{5}_{\text{middle}} + \underbrace{6 \cdot 7}_{\text{right digits}} + \underbrace{3}_{\text{end}} = 95 \text{ modules}.$$

Each quiet zone adds at least nine more module widths. Finding `101` alone proves little: it can also occur inside a digit. The reader tests the complete arrangement.

What does the middle let us do? A slanted scan can leave the top of the bars before reaching the other end. Making the bars taller helps, but costs area.

The middle guard lets each half have two recognizable boundaries:

$$[\,101 \mid \text{six digits} \mid 01010\,], \qquad [\,01010 \mid \text{six digits} \mid 101\,].$$

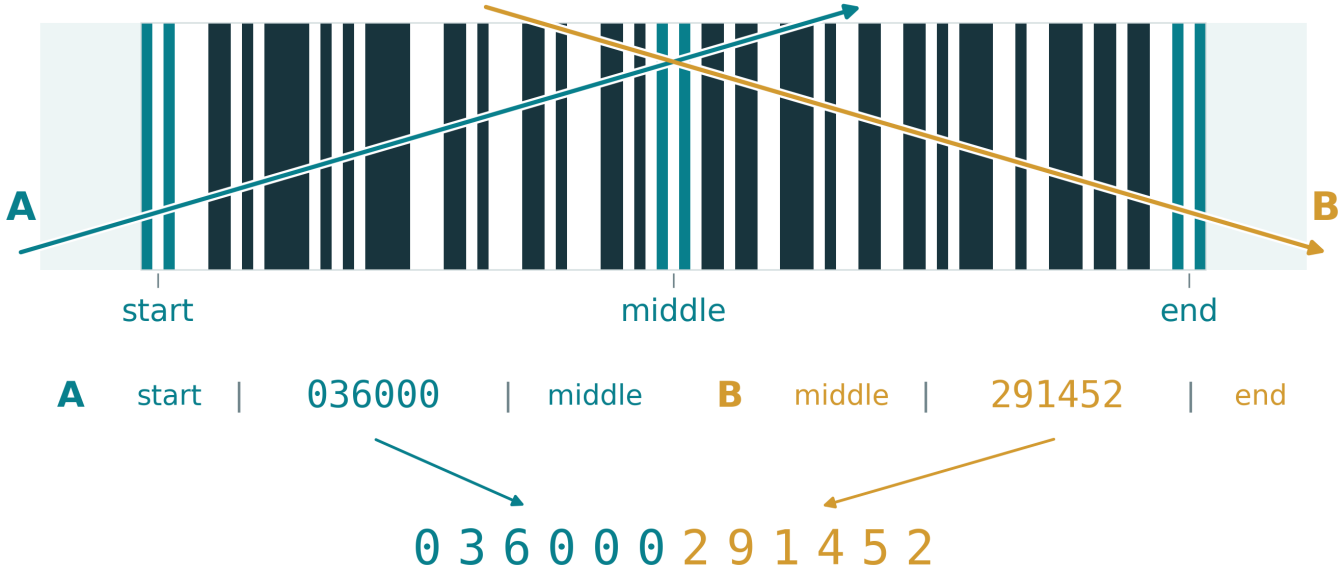


Figure 19: Neither path crosses the whole label. Each crosses one outer guard and the full middle guard, enclosing six digits.

Path A supplies `036000`; path B supplies `291452`. A reader that supports combining half-readings can join compatible halves from the same label:

$$\underbrace{036000}_{\text{left half}} \parallel \underbrace{291452}_{\text{right half}} = \underbrace{036000291452}_{\text{candidate number}}.$$

Here $\parallel$ means join the digit strings. The complete number must still pass its checksum. The middle does not reconstruct missing digits; it makes shorter, bounded readings possible.

How does the reader know which half it has, and which way it crossed the bars?

Read the same pattern in both directions. The guards `101` and `01010` look the same backward. They cannot decide direction by themselves.

For digit 0, the left pattern is $L(0) =$ `0001101`. The right pattern swaps black and white: $R(0) =$ `1110010`. Thus the black-module counts are 3 and $7 - 3 = 4$.

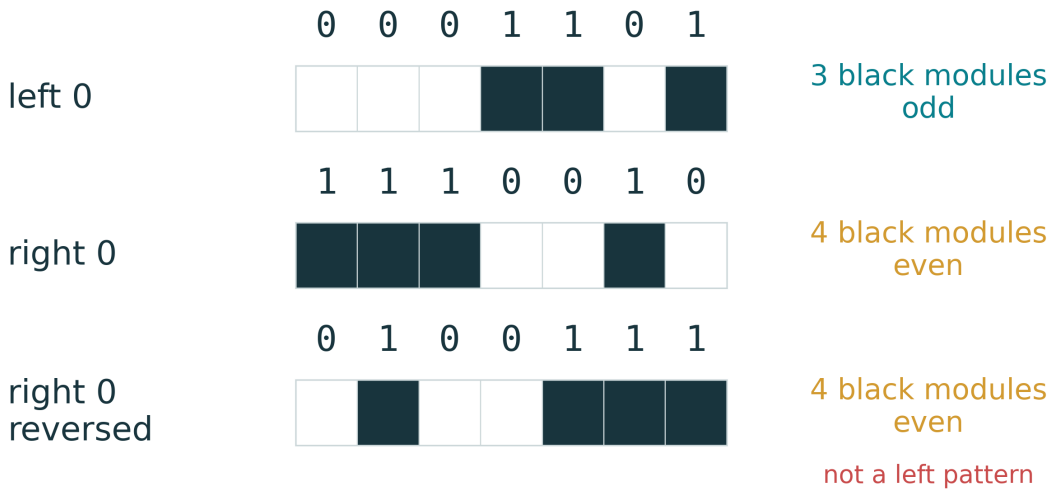


Figure 20: Reading the right-side zero backward gives `0100111`. Its four black modules exclude every left-side digit pattern.

The left and right character-parity tests sum bits modulo two. Reversing positions leaves that sum unchanged. But UPC uses *different required sums on the two sides*:

scan from an outer guard	first six digits	next six digits
left to right	odd	even
right to left	even	odd

For a complete reading, if the even half comes first, reverse the scan before using the normal left/right lookup tables. This reverses both the digit order and the bits within each digit.

What about a half-reading? For a correctly grouped digit, its ends and parity identify four distinct cases:

	first and last bits	black count
left, forward	0 . . . 1	odd
left, backward	1 . . . 0	odd
right, forward	1 . . . 0	even
right, backward	0 . . . 1	even

With valid boundaries and direction, each side's ten patterns have ten different meanings. Parity alone is many-to-one; the full pattern lookup is one-to-one. Damage can still change one allowed pattern into another.

Character parity checks one digit pattern. What rule checks the number across all twelve digits?

7 Check the number, then use it

The plain decimal sum missed swaps:

$$1 + 2 = 2 + 1.$$

Give the two positions different weights:

$$3 \cdot 1 + 2 = 5, \qquad 3 \cdot 2 + 1 = 7.$$

Now that swap changes the check.

UPC-A alternates weights $3, 1$ from the left. For the example we have been reading:



Figure 21: Multiply each digit by the weight beneath it. Gold marks the final check digit.

First add the weighted contributions of the eleven data digits:

$$\underbrace{3(0 + 6 + 0 + 2 + 1 + 5)}_{42} + \underbrace{(3 + 0 + 0 + 9 + 4)}_{16} = 58.$$

Append 2 so that the total is a multiple of ten:

$$58 + \boxed{2} = 60.$$

This is the UPC check-digit rule.

What does this protection cost? There are eleven data digits and one check digit. In this printed format, each uses seven modules:

$$\underbrace{11 \cdot 7}_{\text{77 data modules}} + \underbrace{1 \cdot 7}_{\text{7 check modules}} + \underbrace{3 + 5 + 3}_{\text{11 guard modules}} = 95.$$

The quiet zones require additional blank space. The running remainder $0, \ldots, 9$ fits in four bits. The digit's position determines its weight.

Why not use weight 2? Changing a digit by 5 would change its contribution by $2 \cdot 5 = 10$, which the check would miss. With weight 3, a single digit change Δ contributes 3Δ; with weight 1, it contributes Δ.

For $0 < |\Delta| < 10$, neither is a multiple of ten. Thus every single digit substitution is detected. Some swaps still pass:

$$3 \cdot 1 + 6 = 9, \qquad 3 \cdot 6 + 1 = 19.$$

Their difference is ten, so their remainders agree.

Follow one item through a checkout. A laser scanner sweeps light across the label; an imaging scanner finds paths in an image. The decoder measures widths, tests the frame and digit patterns, and checks the candidate number.



Figure 22: A candidate that passes the tests reaches the product lookup. The product record supplies its price.

If a test fails, try another reading. The LS2208, for example, makes about 100 sweeps per second.[3] A new path may avoid a local defect:

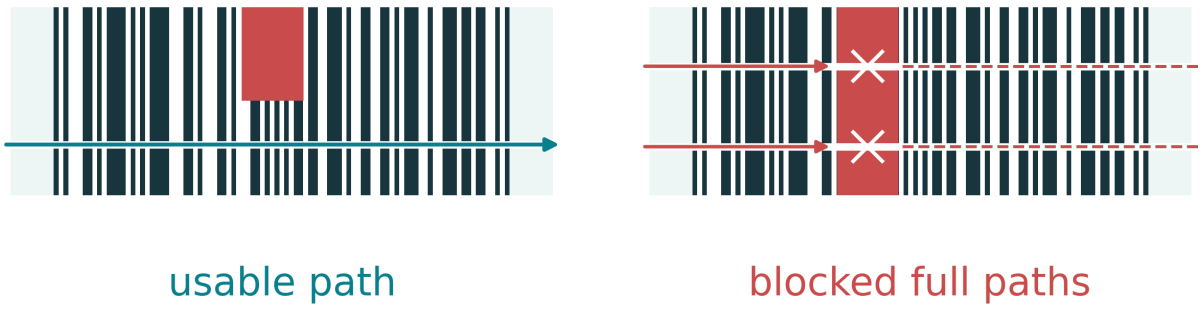


Figure 23: A local scratch leaves a clean path. A full-height obstruction can block every complete path through the symbol.

Some clean area in every column is insufficient. The clean pieces must form a usable path, or provide enough compatible partial readings.

If scanning keeps failing, an operator can reposition the item or enter its printed digits. More attempts help, but they do not guarantee a correct reading. The same damage can cause the same mistake again.

Suppose the check detects an error. Does the failed check also tell us how to repair it?

8 The limit of this check

Two numbers can pass the UPC checksum and still lead to the same rejected reading:



Figure 24: Both original candidates have weighted sum 60. One decimal digit change in either produces the received number, whose sum is 61.

The check detects a problem. Even if we know that exactly one digit changed, it does not identify the original. Choosing either candidate can be wrong.

What if we know which digit is missing? That is a different problem. Suppose only the second digit is unreadable and every other digit is correct:

$$\texttt{0?6000291452}: \qquad 57 + x \equiv 0 \pmod{10} \quad \Longrightarrow \quad x = 3.$$

The known digits contribute 57; the missing digit has weight 1. There is just one choice in $0, \ldots, 9$. So UPC can recover a missing digit under these assumptions. It cannot guarantee correction of one changed digit at an unknown position.

The barcode gives us one working design. Its rules serve different needs: readable stripes, known boundaries, and checks on the digits. Their usefulness depends on the errors we expect and the space we can spend.

Can we describe these rules in a common mathematical framework? What conditions should a design satisfy in different settings? How can we use those conditions to build codes that detect errors, or even let a decoder correct them automatically?

References

[1] Richard W. Hamming. Error Correcting Codes, lecture transcript, Naval Postgraduate School, April 21, 1995, pp. 1–2.

[2] IBM. The UPC. Official history of the barcode's development and first retail use.

[3] Zebra. LS2208 Product Reference Guide, June 2017. Technical specifications and UPC/EAN reading settings.